MTD-based Behavioral Feature Analysis for Bot Detection

This notebook demonstrates how Moving Target Defense (MTD) induces measurable behavioral deviations that can be used for anomaly-based bot detection.

We use:

- Success rate degradation
- Beacon reduction
- DNS behavior

Dataset derived from experimental results.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Step 2: Load Experimental Data

Each row represents one experiment configuration:

- Mode (mutation type)
- Ratio (bot : benign)
- Baseline vs MTD metrics

```
data = [
# Mode, Ratio, Baseline_SR, MTD_SR, Baseline_B, MTD_B, Baseline_DNS, MTD_DNS, Beacon_Reduction, Attack_Suppression

["dns","20:80",94.441,89.036,2306.25,2177,12169.4,12352.5,0.006,-0.001],
["dns","50:5",96.991,96.82,5878.5,5878.25,6668.88,6670.5,-0.004,0.001],
["dns","50:50",94.626,73.504,5760.75,4505.625,12189.5,13008.75,0.222,0.228],

["ip","20:80",94.441,39.116,2306.25,820.5,12169.4,10530.25,0.625,0.56],
["ip","50:5",96.991,39.934,5878.5,2089.5,6668.88,5778.5,0.643,0.588],
["ip","50:50",94.626,39.201,5760.75,2055.375,12189.5,10524.75,0.645,0.588],

["ip_dns","20:80",94.441,39.212,2306.25,820.625,12169.4,10527.13,0.625,0.559],
["ip_dns","50:5",96.991,39.805,5878.5,2083.75,6668.88,5784,0.644,0.589],
["ip_dns","50:50",94.626,39.16,5760.75,2054,12189.5,10529.75,0.645,0.589],

["ip_port","20:80",94.441,11.774,2306.25,268.25,12169.4,11435.5,0.877,0.868],
["ip_port","50:5",96.991,12.336,5878.5,700.375,6668.88,6265,0.88,0.873],
["ip_port","50:50",94.626,11.661,5760.75,665.125,12189.5,11423.38,0.885,0.878],

["ip_port_dns","20:80",94.441,12.196,2306.25,277.125,12169.4,11399.13,0.873,0.863],
["ip_port_dns","50:5",96.991,11.728,5878.5,668.625,6668.88,6288.125,0.886,0.879],
["ip_port_dns","50:50",94.626,11.696,5760.75,666,12189.5,11414,0.885,0.877],
]

columns = ["Mode","Ratio","Baseline_SR","MTD_SR","Baseline_B","MTD_B","Baseline_DNS","MTD_DNS","Beacon_Reduction","Attack_Su

df = pd.DataFrame(data, columns=columns)
df.head()
```

	Mode	Ratio	Baseline_SR	MTD_SR	Baseline_B	MTD_B	Baseline_DNS	MTD_DNS	Beacon_Reduction	Attack_Suppression
0	dns	20:80	94.441	89.036	2306.25	2177.000	12169.40	12352.50	0.006	-0.001
1	dns	50:5	96.991	96.820	5878.50	5878.250	6668.88	6670.50	-0.004	0.001
2	dns	50:50	94.626	73.504	5760.75	4505.625	12189.50	13008.75	0.222	0.228
3	ip	20:80	94.441	39.116	2306.25	820.500	12169.40	10530.25	0.625	0.560
4	ip	50:5	96.991	39.934	5878.50	2089.500	6668.88	5778.50	0.643	0.588

Step 3: Feature Engineering

We derive behavioral features that capture disruption caused by MTD:

- SR_drop → drop in connection success
- Beacon_drop → drop in command delivery
- DNS_diff → DNS behavior change

- FRc → failure rate approximation

These features form the ML-ready feature space.

```
df["SR_drop"] = df["Baseline_SR"] - df["MTD_SR"]
df["Beacon_drop"] = df["Baseline_B"] - df["MTD_B"]
df["DNS_diff"] = df["MTD_DNS"] - df["Baseline_DNS"]
df["FRc"] = 1 - (df["MTD_SR"] / 100)

df[["Mode","Ratio","SR_drop","Beacon_drop","DNS_diff","Attack_Suppression"]]
```

	Mode	Ratio	SR_drop	Beacon_drop	DNS_diff	Attack_Suppression
0	dns	20:80	5.405	129.250	183.100	-0.001
1	dns	50:5	0.171	0.250	1.620	0.001
2	dns	50:50	21.122	1255.125	819.250	0.228
3	ip	20:80	55.325	1485.750	-1639.150	0.560
4	ip	50:5	57.057	3789.000	-890.380	0.588
5	ip	50:50	55.425	3705.375	-1664.750	0.588
6	ip_dns	20:80	55.229	1485.625	-1642.270	0.559
7	ip_dns	50:5	57.186	3794.750	-884.880	0.589
8	ip_dns	50:50	55.466	3706.750	-1659.750	0.589
9	ip_port	20:80	82.667	2038.000	-733.900	0.868
10	ip_port	50:5	84.655	5178.125	-403.880	0.873
11	ip_port	50:50	82.965	5095.625	-766.120	0.878
12	ip_port_dns	20:80	82.245	2029.125	-770.270	0.863
13	ip_port_dns	50:5	85.263	5209.875	-380.755	0.879
14	ip_port_dns	50:50	82.930	5094.750	-775.500	0.877

Step 4: SR Drop vs Beacon Drop

This plot represents the degree of communication disruption under MTD.

- SR_drop: reduction in successful connections
- Beacon_drop: reduction in successful C2 command delivery

Interpretation:

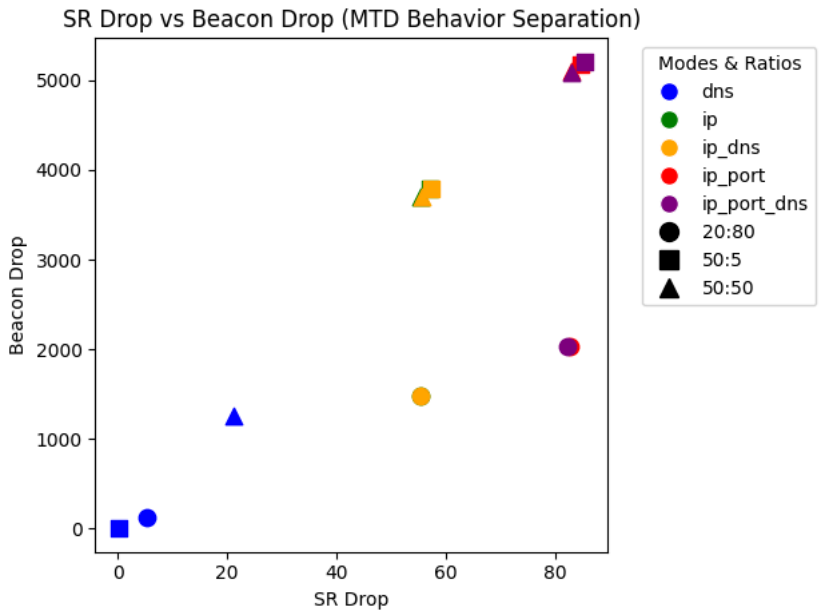
- Low SR_drop and low Beacon_drop → stable communication (benign-like)
- High SR_drop and high Beacon_drop → strong disruption (bot-like behavior)
- Partial increases indicate intermediate disruption levels

Points farther from the origin represent higher communication instability.

This demonstrates that MTD induces measurable behavioral changes that can be used for anomaly-based detection.

```
plt.figure()

# Color per mode
colors = {
    "dns": "blue",
    "ip": "green",
    "ip_dns": "orange",
    "ip_port": "red",
    "ip_port_dns": "purple"
}

# Marker per ratio
markers = {
    "20:80": "o",
    "50:5": "s",
    "50:50": "^"
}

# Plot each point
for _, row in df.iterrows():
    plt.scatter(
        row["SR_drop"],
        row["Beacon_drop"],
        color=colors[row["Mode"]],
```

```
        marker=markers[row["Ratio"]],
        s=80
    )

# Create legend manually
from matplotlib.lines import Line2D

mode_legend = [
    Line2D([0], [0], marker='o', color='w', label=mode,
           markerfacecolor=color, markersize=10)
    for mode, color in colors.items()
]

ratio_legend = [
    Line2D([0], [0], marker=marker, color='black', label=ratio,
           linestyle='None', markersize=10)
    for ratio, marker in markers.items()
]

plt.legend(handles=mode_legend + ratio_legend, title="Modes & Ratios", bbox_to_anchor=(1.05, 1), loc='upper left')

plt.xlabel("SR Drop")
plt.ylabel("Beacon Drop")
plt.title("SR Drop vs Beacon Drop (MTD Behavior Separation)")
plt.tight_layout()
plt.show()
```



Step 5: SR Drop vs Attack Suppression

This plot evaluates consistency and strength of disruption across MTD strategies.

- X-axis (SR_drop): reduction in connection success
- Y-axis (Attack_Suppression): overall disruption effectiveness

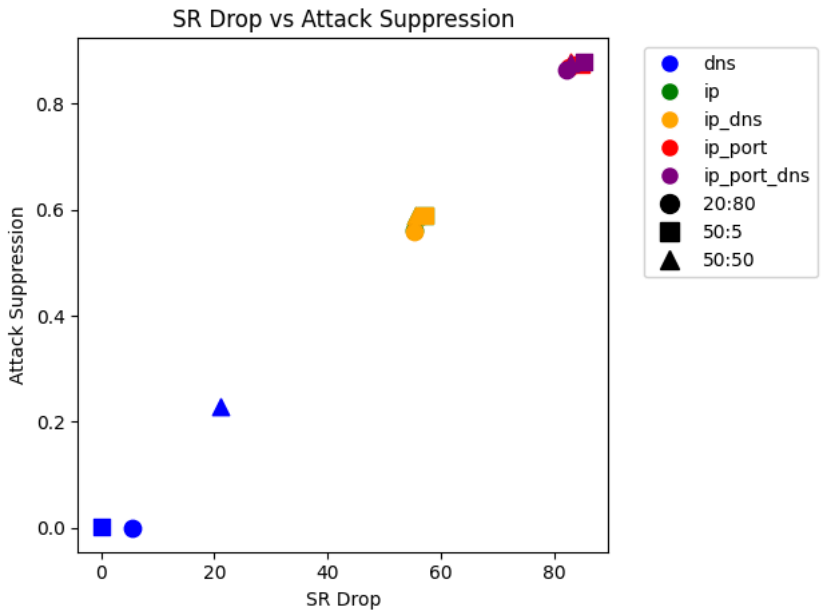
Interpretation:

- Points near origin → weak disruption
- Points in upper-right → strong disruption
- Clear grouping across strategies indicates consistent behavioral patterns

This demonstrates that MTD creates a structured and separable feature space suitable for ML-based detection.

```
plt.figure()

for _, row in df.iterrows():
    plt.scatter(
        row["SR_drop"],
        row["Attack_Suppression"],
        color=colors[row["Mode"]],
        marker=markers[row["Ratio"]],
        s=80
    )

from matplotlib.lines import Line2D
```

```
mode_legend = [
    Line2D([0], [0], marker='o', color='w', label=mode,
           markerfacecolor=color, markersize=10)
    for mode, color in colors.items()
]

ratio_legend = [
    Line2D([0], [0], marker=marker, color='black', label=ratio,
           linestyle='None', markersize=10)
    for ratio, marker in markers.items()
]

plt.legend(handles=mode_legend + ratio_legend,
           bbox_to_anchor=(1.05, 1), loc='upper left')

plt.xlabel("SR Drop")
plt.ylabel("Attack Suppression")
plt.title("SR Drop vs Attack Suppression")
plt.tight_layout()
plt.show()
```



Step 6: Unsupervised Behavioral Clustering

To evaluate whether MTD-induced behavior forms natural groupings into Stable behavior (benign-like) and Disrupted behavior (bot-like), we apply KMeans clustering using:

- SR_drop (connection degradation)
- Beacon_drop (C2 communication degradation)

This demonstrates that MTD creates an inherent structure in the feature space suitable for anomaly detection.

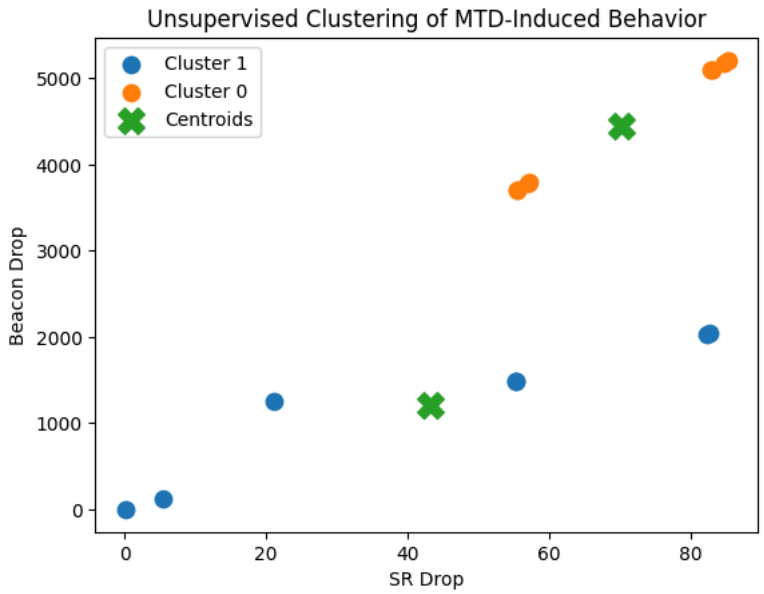
Interpretation:

- Cluster near origin → low disruption → stable communication
- Cluster far from origin → high disruption → unstable communication

This supports the hypothesis that MTD-induced instability can serve as a detection signal; And shows which strategies produce strong vs weak behavioral separation.

```
X = df[["SR_drop","Beacon_drop"]]

kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)

plt.figure()

# plot clusters
for cluster in df["Cluster"].unique():
    subset = df[df["Cluster"] == cluster]
    plt.scatter(subset["SR_drop"], subset["Beacon_drop"], label=f"Cluster {cluster}", s=80)

# plot centroids
centroids = kmeans.cluster_centers_
```

```
plt.scatter(centroids[:,0], centroids[:,1], marker='X', s=200, label='Centroids')

plt.xlabel("SR Drop")
plt.ylabel("Beacon Drop")
plt.title("Unsupervised Clustering of MTD-Induced Behavior")
plt.legend()
plt.show()
```



Conclusion

MTD induces measurable communication instability, reflected in features like success rate and beacon reduction. These features show clear separation between stable and disrupted behavior, which is further validated through clustering.

This demonstrates that MTD not only disrupts botnet communication but also creates a structured feature space that can support anomaly-based detection without requiring labeled data.

Double-click (or enter) to edit